

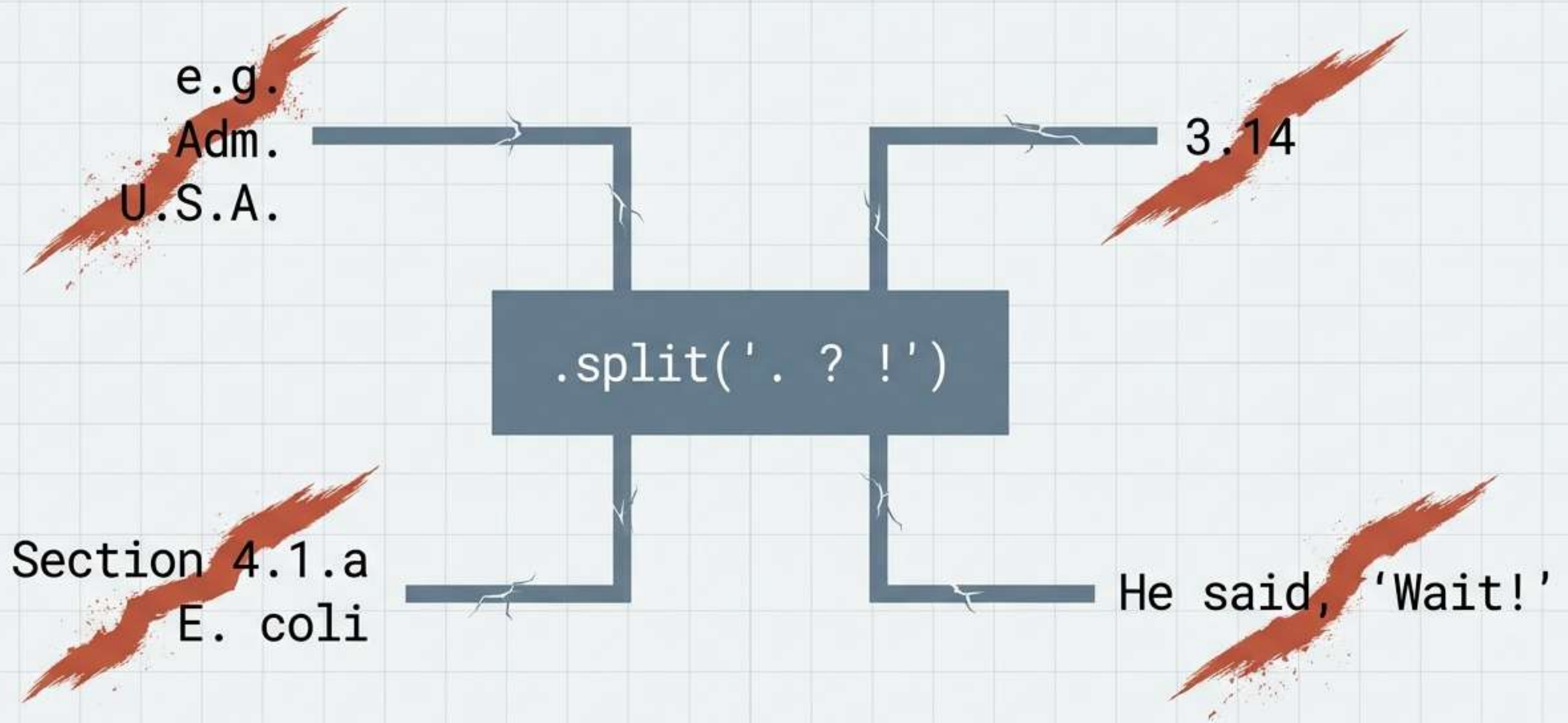
# The Sentence Boundary Paradox

## Moving from Text Mutation to Query Planning

**The U.S.A. budget (approx.  
\$3.14M) was approved! [111]**

[0:56]

# The myth of `.split()`



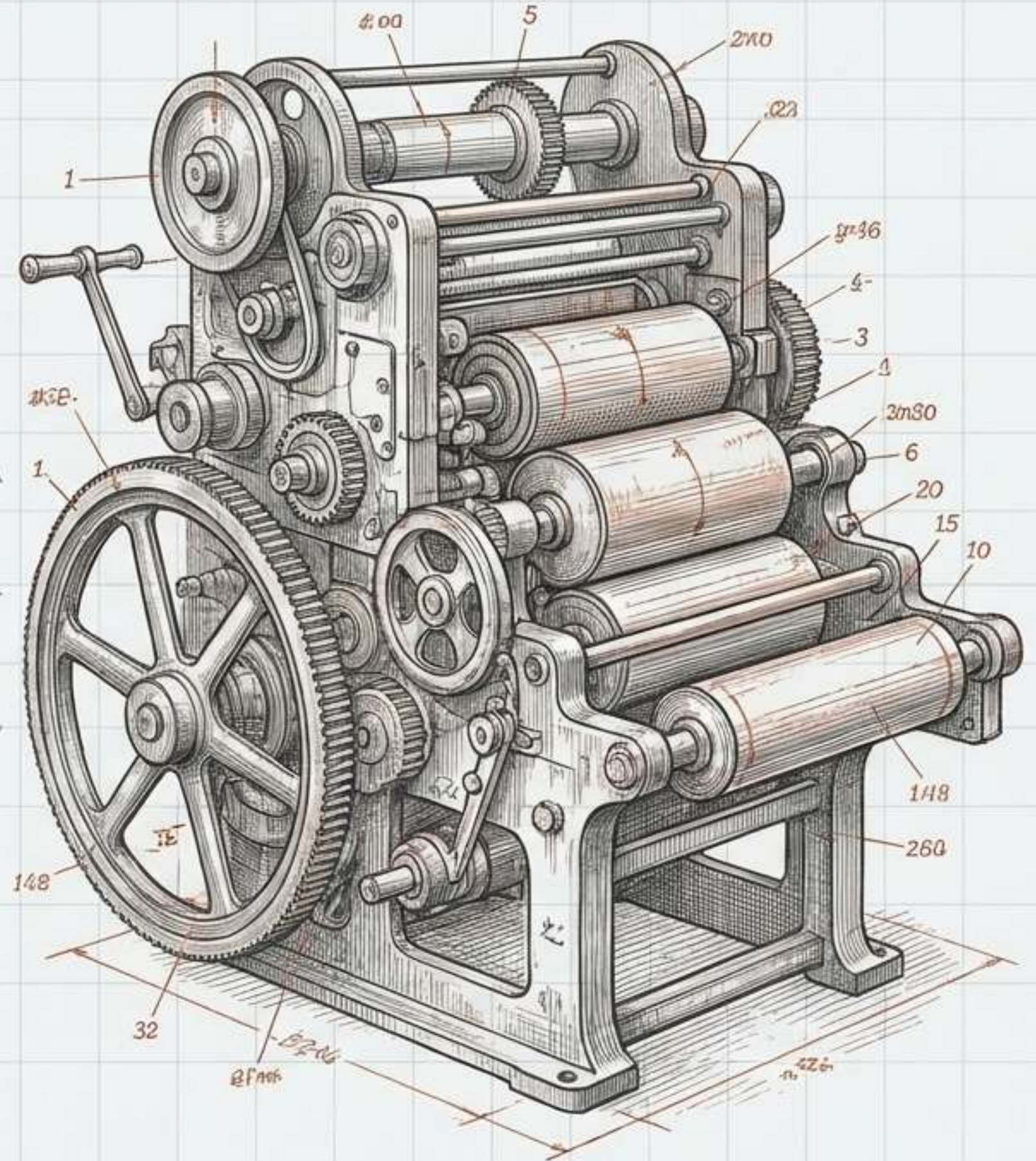
Sentence boundary detection is not a string operation; it is a language-specific parsing puzzle.

# 2020: The pragmatic segmentation standard

**Origin:** Ported directly from Ruby's `pragmatic_segmenter`.

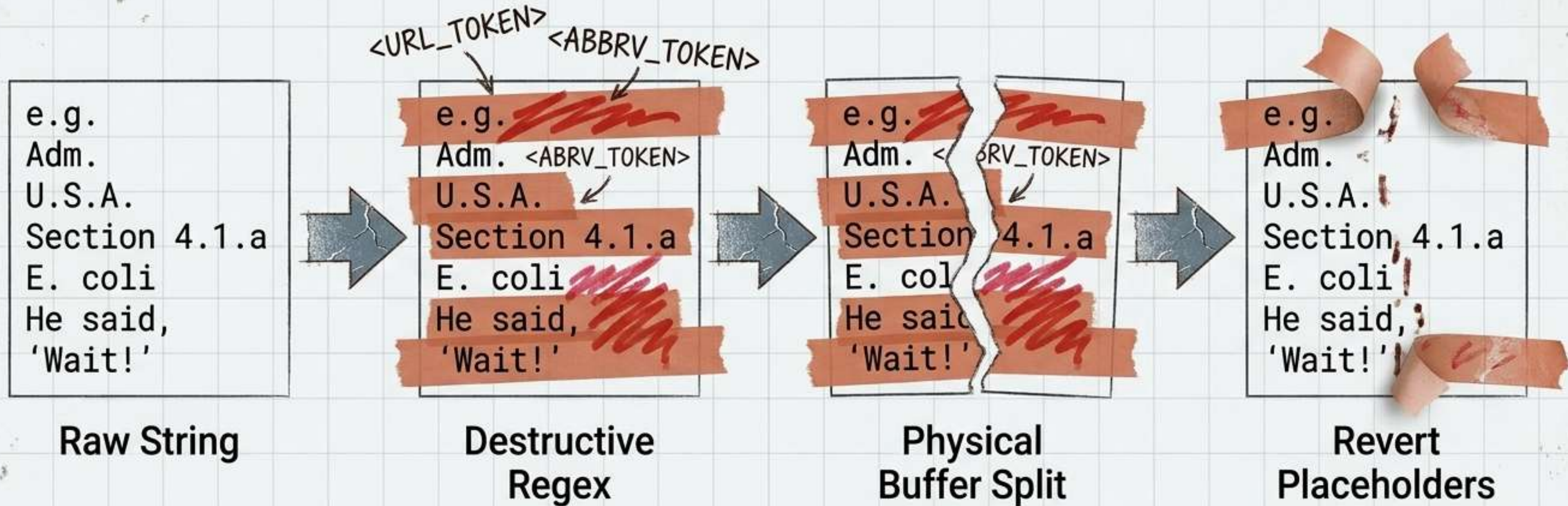
**Scale:** Out-of-the-box support for 22 languages.

**Achievement:** Passed 97.92% of the original Golden Rule Set exemplars.



For years, PySBD served as the baseline for unstructured text processing, functioning as a robust, albeit monolithic, multi-stage pipeline.

# PySBD Architecture: The transformation pipeline



This pipeline treats text as a mutable object—destroying the original string layout mid-flight to protect inner punctuation from false splits.

# The span reconstruction trap

Because PySBD modifies text mid-flight, it loses track of original character coordinates. To calculate exact character offsets (spans), it must execute a post-processing search over the **entire original document for every single sentence.**

**In documents with repeated sentences, this scanning approaches near-quadratic  $O(N^2)$  computational overhead.**

# THE BREAKING POINT: UNMAINTAINED AND UNSTABLE

ISSUE #79 (OCT 2020)

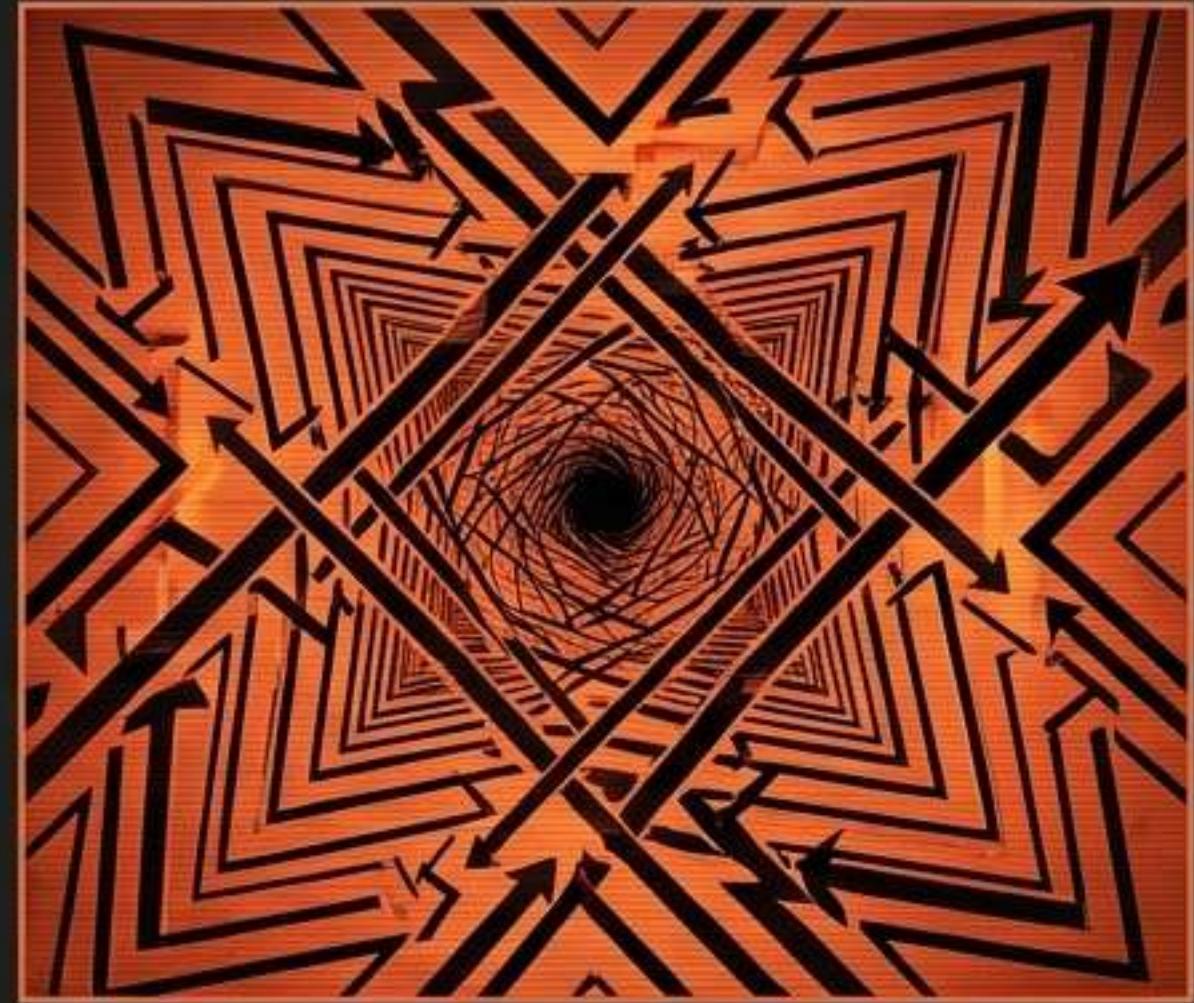
NUMBERED\_REFERENCE\_REGEX



Crashing indefinitely on inputs like [111 111 111...]

ISSUE #92 (FEB 2021)

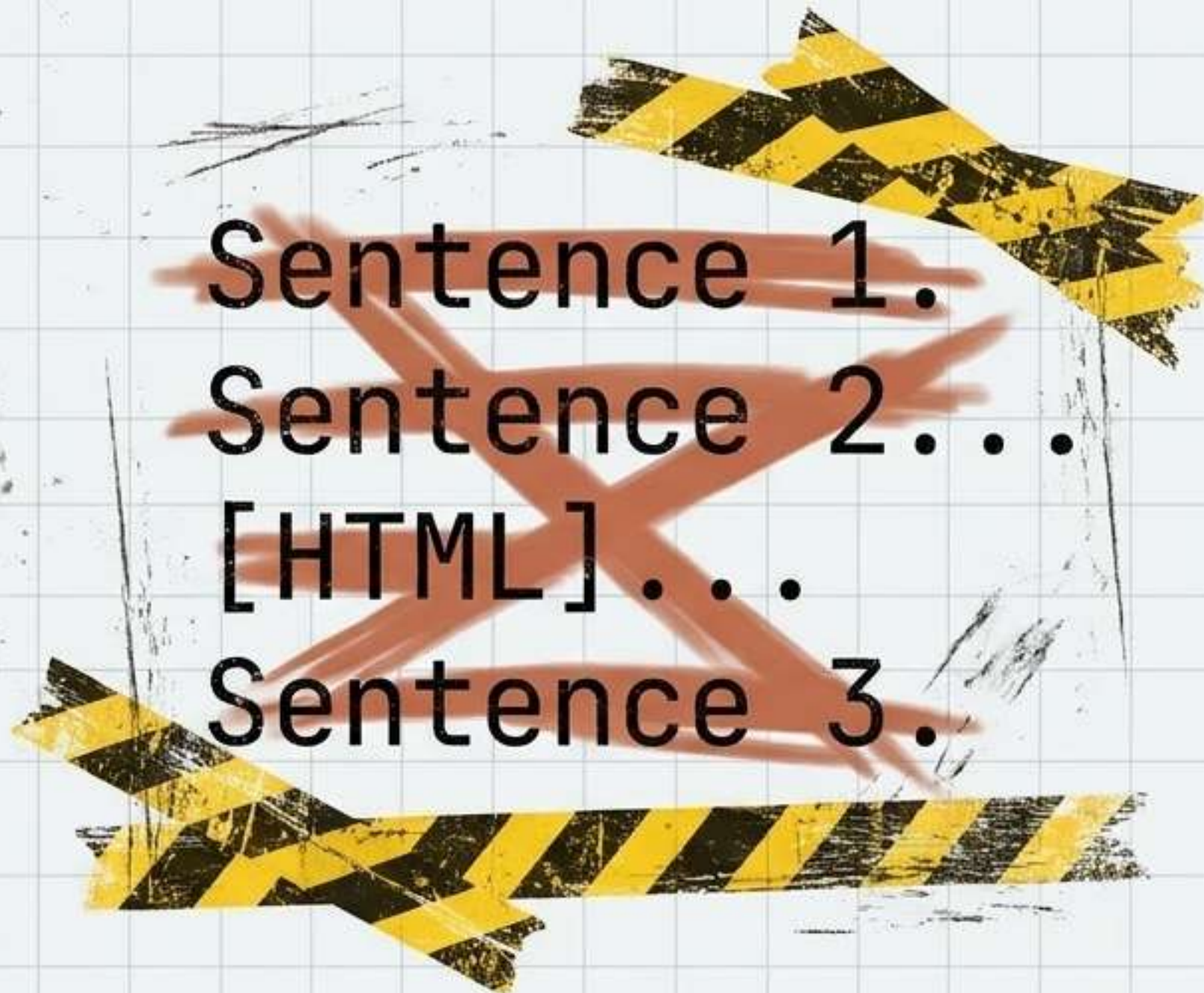
HTMLTagRule



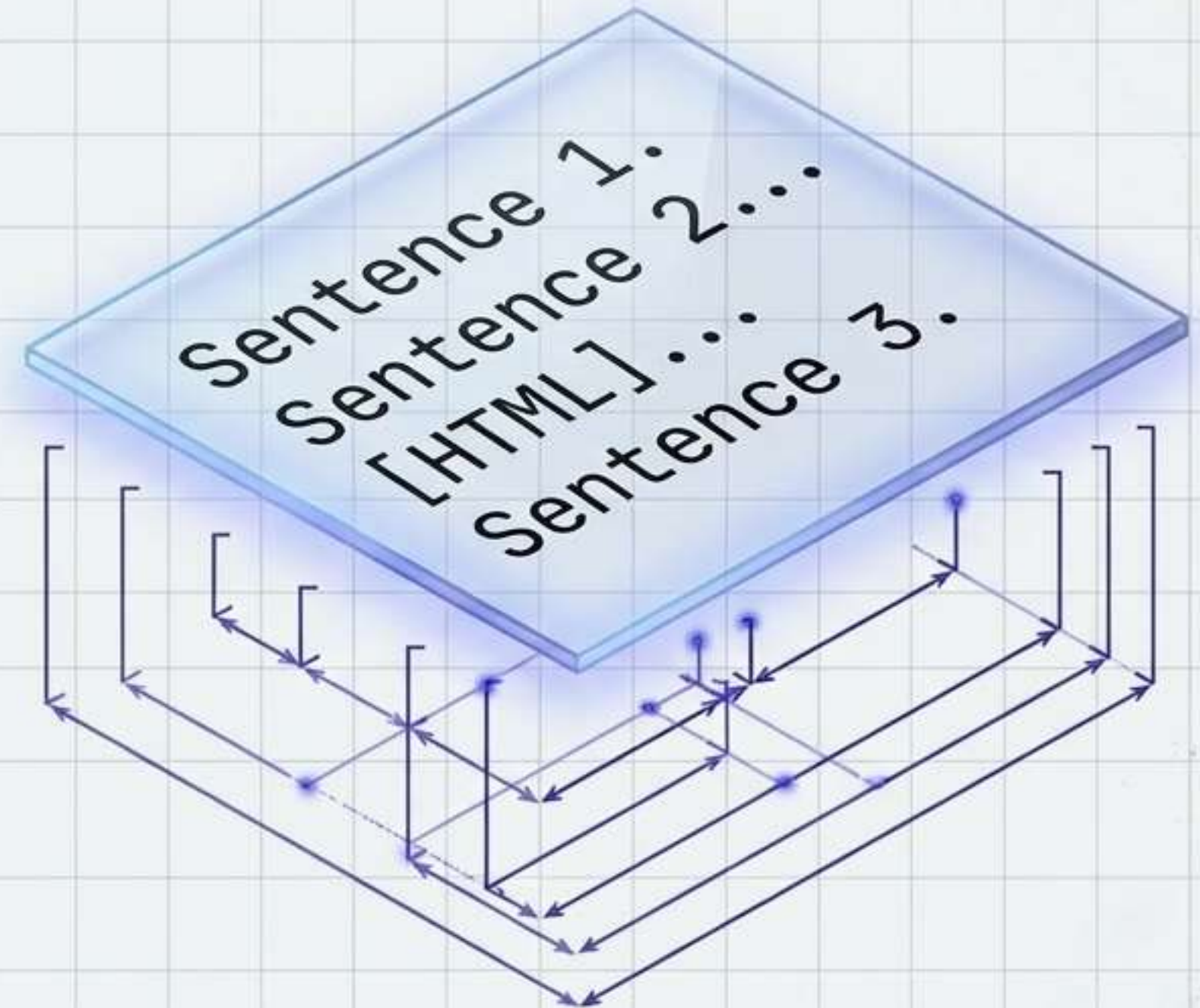
Hanging indefinitely on unfinished HTML attributes.

**AS OF 2025, PYSBD IS EFFECTIVELY ABANDONED. ITS RELIANCE ON NESTED REGEX QUANTIFIERS WITHIN A SEQUENTIAL TRANSFORMATION PIPELINE CREATES UNRESOLVABLE INFINITE LOOPS.**

# We must stop protecting boundaries and start finding them

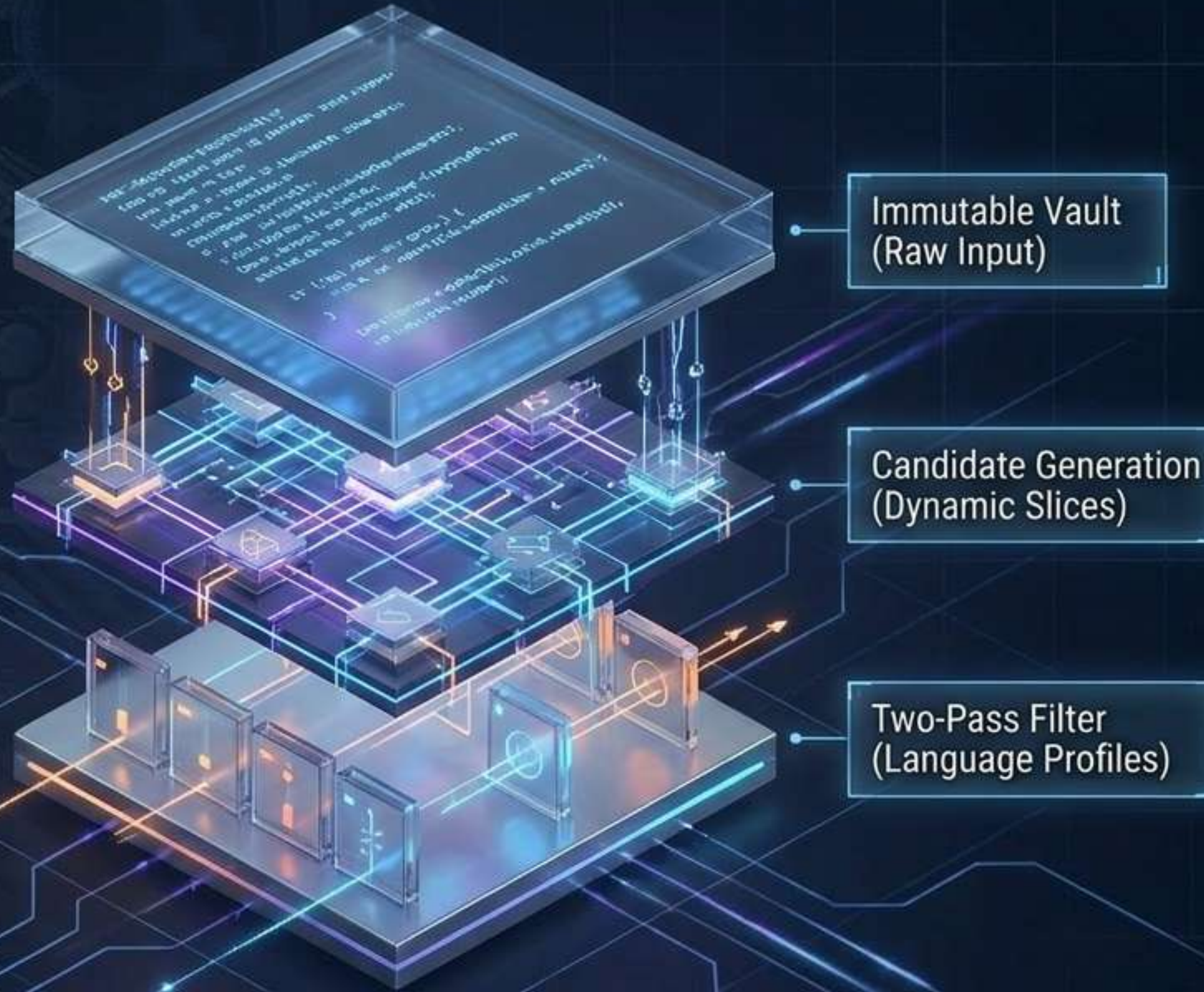


**Legacy approach:  
String Mutation**



**yasbd-lib approach:  
Structural Cartography**

# Enter yasbd-lib: The query planning approach



- Expanded to 39 languages out-of-the-box.
- Zero string mutation.
- Evolves integer pointers rather than altering buffers.

# Spans as a first-class structural signal

The engine initialized. (System nominal.)

`text[start:end]`

## No Lookback

Because strings are never mutated, token spans are tracked natively during parsing without secondary document searches.

## Preservation Logic

Built-in pointer rules seamlessly manage bounds inside parentheses with `preserve_quote_and_paren=True`.

## Result

Absolute coordinate fidelity for downstream NER models and precise RAG indexing.

# The Sherlock Holmes benchmark (594k characters)

PySBD



13.3 seconds

yasbd-lib



1.6 seconds

The pointer-based architecture avoids  $O(N^2)$  post-processing friction. It processes text continuously rather than modifying, transforming, and rescanning complete string buffers.

# Memory constraints and stream processing

ParagraphStream

StreamCleaner



The diagram illustrates a stream processing pipeline. A series of parallel lines representing a data stream enters from the left. It first passes through a vertical rectangular filter labeled 'StreamCleaner'. After exiting this filter, the stream enters a second, similar vertical rectangular filter labeled 'ParagraphStream'. The stream continues to the right after passing through the second filter. A warning triangle icon is visible in the upper right background.

Safely unwraps HTML  
without the nested quantifiers  
quantifiers that cause  
legacy systems to hang.

Lazy-loads blocks of text,  
keeping the application memory  
footprint entirely flat.

Engine Mechanics:

- LRU cache execution (`_MAX_CACHED_RULES = 5`)
- Fail-safe confidence logging (`_MIN_CONFIDENCE = 0.8`)

# Dominating an expanded golden standard

PySBD



yasbd-lib

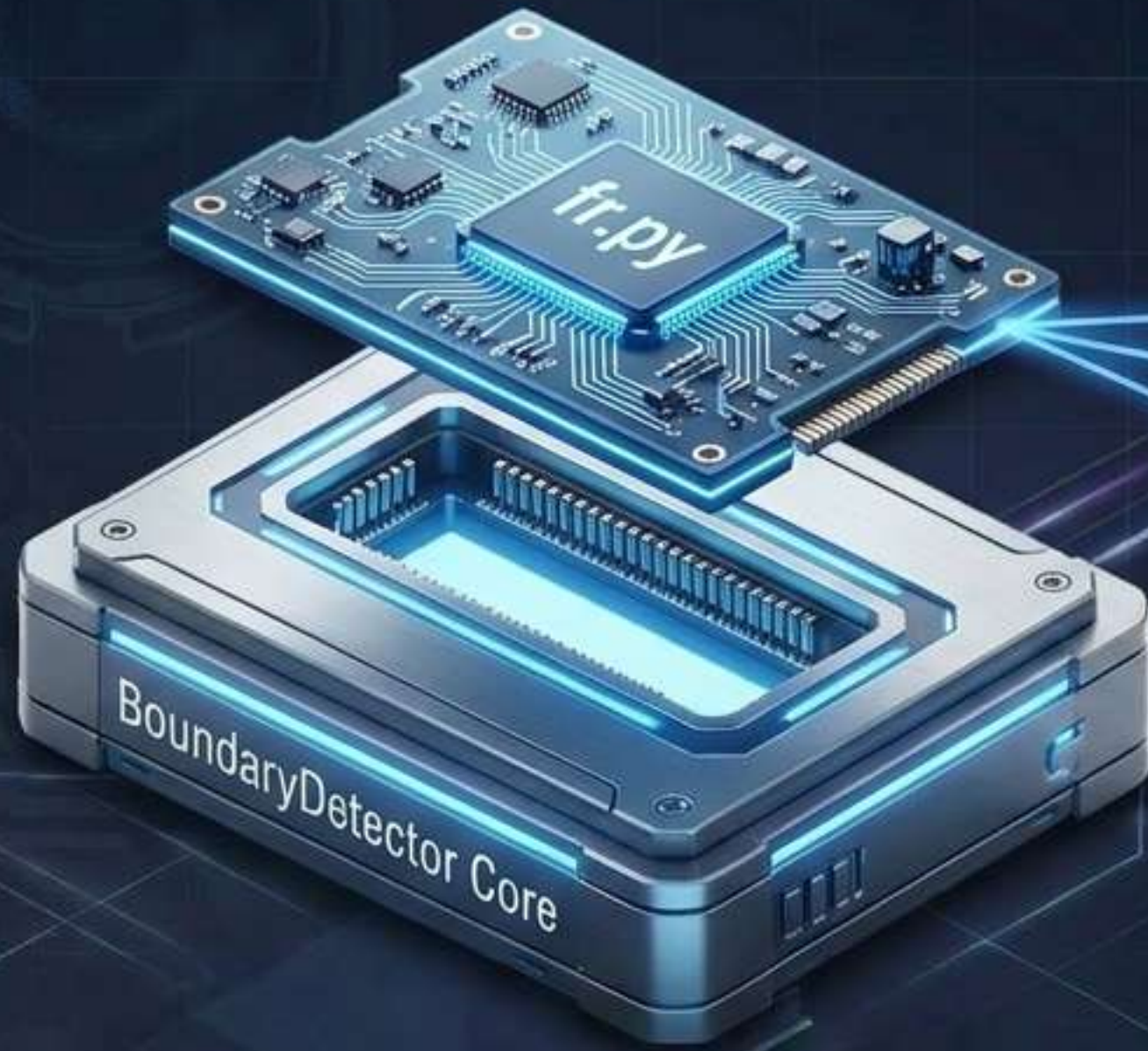


The evaluation corpus was expanded from 48 to 92 rigorous edge cases—adding abbreviation chains, contiguous terminators, and markdown formatting. The two-pass boundary detection proves drastically more accurate than sequential regex modification.

# The architectural diagnostic

| Feature           | PySBD                                      | yasbd-lib                  |
|-------------------|--------------------------------------------|----------------------------|
| Approach          | Monolithic transformation                  | Modular immutable pipeline |
| State Handling    | String mutation & placeholders             | Pointer-based operations   |
| Language Profiles | 22                                         | 39                         |
| Character Spans   | $O(N^2)$ reconstruction                    | Native dynamic tracking    |
| Known Regex Risks | Infinite loops & catastrophic backtracking | Safe two-pass execution    |
| Maintenance       | Abandoned                                  | Active                     |

# Extensibility via declarative language profiles



TITLE\_ABBRVS

TERMINATORS

POST\_QUOTATIVE\_PARTICLES

Legacy rules were deeply interdependent. yasbd-lib uses a decoupled, declarative profile system. Changes to a language-specific profile do not alter the core engine's boundary detection mechanics. Custom packs load dynamically via `register_lang_packs()`.

# A seamless architectural bridge



```
pip install yasbd-lib
```



## Risk-Free Migration

Features a drop-in adapter layer for immediate, risk-free migration from PySBD.



## Modern Integration

Native integration directly into modern spaCy v3+ custom pipelines.



## Enhanced Stability

Eliminates infinite loop risks instantly without rewriting any upstream logic.

*Shift from text mutation to query planning today.*